Programação e Orientação a Objetos

Leonardo Vitazik Neto

Católica de Santa Catarina
Centro Universitário

Agenda

- Variáveis primitivas;
- Java code conventions;
- Casting e promoção;
- Material complementar:
  - Java;
  - Máquina virtual;
- Exercício.

VARIÁVEIS PRIMITIVAS

DECLARANDO E USANDO VARIÁVEIS

Dentro de um bloco, podemos declarar variáveis e usá-las. Em Java, toda variável tem um tipo que não pode ser mudado, uma vez que declarado:

```
tipoDaVariavel nomeDaVariavel;
```

Por exemplo, é possível ter uma idade que guarda um número inteiro:

```
int idade;
```

Com isso, você declara a variável idade , que passa a existir a partir daquela linha. Ela é do tipo int , que guarda um número inteiro. A partir daí, você pode usá-la, primeiramente atribuindo valores.
A linha a seguir é a tradução de: " **idade deve valer quinze**".

```
idade = 15;
```

USANDO VARIÁVEIS

Além de atribuir, você pode utilizar esse valor. O código a seguir declara novamente a variável idade com valor 15 e imprime seu valor na saída padrão através da chamada a **System.out.println**.

```
// declara a idade
int idade;
idade = 15;

// imprime a idade
System.out.println(idade);
```

COMENTÁRIOS EM JAVA

Para fazer um comentário em java, você pode usar o // para comentar até o final da linha, ou então usar o /* */ para comentar o que estiver entre eles.

```
 /* comentário daqui,
ate aqui */

// uma linha de comentário sobre a idade
int idade;
```

USANDO VARIÁVEIS

Por fim, podemos utilizar o valor de uma variável para algum outro propósito, como alterar ou definir uma segunda variável. O código a seguir cria uma variável chamada **idadeNoAnoQueVem** com valor de idade mais um.

```
// calcula a idade no ano seguinte
int idadeNoAnoQueVem;
idadeNoAnoQueVem = idade + 1;
```

USANDO VARIÁVEIS

No mesmo momento que você declara uma variável, também é possível inicializá-la por praticidade:

```
int idade = 15;
```

Você pode usar os operadores +, -, / e * para operar com números, sendo eles responsáveis pela adição, subtração, divisão e multiplicação, respectivamente. Além desses operadores básicos, há o operador % (módulo) que nada mais é que o resto de uma divisão inteira. Veja alguns exemplos:

```
int quatro = 2 + 2;
int tres = 5 - 2;

int oito = 4 * 2;
int dezesseis = 64 / 4;

int um = 5 % 2; // 5 dividido por 2 dá 2 e tem resto 1;
                // o operador % pega o resto da divisão inteira
```

COMO RODAR ESSES CÓDIGOS?

Você deve colocar esses trechos de código dentro do bloco main que vimos no capítulo anterior.
Isto é, isso deve ficar no miolo do programa. Use bastante System.out.println , dessa forma você pode ver algum resultado, caso contrário, ao executar a aplicação, nada aparecerá.

Por exemplo, para imprimir a idade e a **idadeNoAnoQueVem** podemos escrever o seguinte programa de exemplo:

```
class TestaIdade {

    public static void main(String[] args) {
        // imprime a idade
        int idade = 20;
        System.out.println(idade);

        // gera uma idade no ano seguinte
        int idadeNoAnoQueVem;
        idadeNoAnoQueVem = idade + 1;

        // imprime a idade
        System.out.println(idadeNoAnoQueVem);
    }
}
```

USANDO VARIÁVEIS

Representar números inteiros é fácil, mas como guardar valores reais, tais como frações de números inteiros e outros? Outro tipo de variável muito utilizado é o double , que armazena um número com ponto flutuante (e que também pode armazenar um número inteiro).

```
double pi = 3.14;
double x = 5 * 10;
```

O tipo boolean armazena um valor verdadeiro ou falso, e só: nada de números, palavras ou endereços, como em algumas outras linguagens.

```
boolean verdade = true;
```

true e false são palavras reservadas do Java. É comum que um boolean seja determinado através de uma expressão booleana, isto é, um trecho de código que retorna um booleano, como o exemplo:

```
int idade = 30;
boolean menorDeIdade = idade < 18;
```

USANDO VARIÁVEIS

O tipo char guarda um, e apenas um, caractere. Esse caractere deve estar entre aspas simples. Não se esqueça dessas duas características de uma variável do tipo char! Por exemplo, ela não pode guardar um código como '' pois o vazio não é um caractere!

```
char letra = 'a';
```

Variáveis do tipo char são pouco usadas no dia a dia. Veremos mais a frente o uso das String s, que usamos constantemente, porém estas não são definidas por um tipo primitivo.

TIPOS PRIMITIVOS E VALORES

Esses tipos de variáveis são tipos primitivos do Java: o valor que elas guardam são o real conteúdo da variável. Quando você utilizar o operador de atribuição = o valor será copiado.

```
int i = 5; // i recebe uma cópia do valor 5
int j = i; // j recebe uma cópia do valor de i
i = i + 1; // i vira 6, j continua 5
```

Aqui, i fica com o valor de 6. Mas e j ? Na segunda linha, j está valendo 5. Quando i passa a valer 6, será que j também muda de valor? Não, pois o valor de um tipo primitivo sempre é copiado.

Apesar da linha 2 fazer j = i , a partir desse momento essas variáveis não tem relação nenhuma: o que acontece com uma, não reflete em nada com a outra.

java code conventions

https://www.oracle.com/java/technologies/javase/codeconventions-contents.html

java code conventions

Ao usar as convenções de nomenclatura padrão do Java, você facilita a leitura do código para você e outros programadores. A legibilidade do programa Java é muito importante. Isso indica que menos tempo é gasto para descobrir o que o código faz.

A seguir, estão as principais regras que devem ser seguidas por todos os identificadores:

- O nome não deve conter espaços em branco.
- O nome não deve começar com caracteres especiais como & (e comercial), $ (dólar), _ (sublinhado).

Classe

Conceito: Em Java (e em muitas outras linguagens de programação orientadas a objetos), uma classe é um modelo ou blueprint a partir do qual objetos são criados. Ela define um conjunto de propriedades (atributos) e comportamentos (métodos) que os objetos dessa classe terão.

Convenção

- Deve começar com a letra maiúscula.
- Deve ser um substantivo como Cor, Botão, Sistema, Tópico, etc.
- Use palavras apropriadas, em vez de siglas.

```
public class Employee {
    //fragmento de código
}
```

Método

Conceito: São as funções ou operações que a classe pode realizar. Eles geralmente operam nos atributos da classe ou executam ações relacionadas a esses atributos.
No caso da classe Car, métodos poderiam ser acelerar(), frear(), etc.

Convenção

- Deve começar com letra minúscula.
- Deve ser um verbo como main(), print(), println().
- Se o nome contiver várias palavras, inicie-o com uma letra minúscula seguida por uma letra maiúscula, como actionPerformed().

```
class Employee {
    //método
    void desenhar() {
        //fragmento de código
    }
}
```

Variável

Conceito: São as características (propriedades) ou dados que a classe armazena.
Por exemplo, se você tem uma classe Car, os atributos podem ser cor, marca, modelo, etc.

Convenção

- Ele deve começar com uma letra minúscula, como id, nome.
- Não deve começar com caracteres especiais como & (e comercial), $ (dólar), _ (sublinhado).
- Se o nome contiver várias palavras, inicie-o com a letra minúscula seguida por uma letra maiúscula como firstName, lastName.
- Evite usar variáveis de um caractere, como x, y, z.

```
class Employee {
    //variável
    int  id;
    //fragmento de código
}
```

Pacote

- Deve ser uma letra minúscula, como java, lang.
- Se o nome contiver várias palavras, ele deverá ser separado por pontos (.) Como java.util, java.lang.

```
package com.javatpoint; //pacote
class Employee {
   //fragmento de código
}
```

Constante

- Deve estar em letras maiúsculas, como VERMELHO, AMARELO.
- Se o nome contiver várias palavras, ele deverá ser separado por um sublinhado (_), como MAX_PRIORITY.
- Pode conter dígitos, mas não como a primeira letra.

```
class Employee {
  //constante&
  final static int MIN_AGE = 18 ;
  //fragmento de código
}
```

CamelCase em convenções de nomenclatura java

Java segue a sintaxe **camelCase** para nomear a classe, interface, método e variável.

Se o nome for combinado com duas palavras, a segunda palavra começará com letras maiúsculas sempre como actionPerformed(), firstName, ActionEvent, ActionListener, etc.

CASTING E PROMOÇÃO

Alguns valores são incompatíveis se você tentar fazer uma atribuição direta. Enquanto um número real costuma ser representado em uma variável do tipo double , tentar atribuir ele a uma variável int não funciona porque é um código que diz: "i deve valer d", mas não se sabe se d realmente é um número inteiro ou não.

```
double d = 3.1415;
int i = d; // não compila
```

O mesmo ocorre no seguinte trecho:

```
int i = 3.14;
```

O mais interessante, é que nem mesmo o seguinte código compila:

```
double d = 5; // ok, o double pode conter um número inteiro
int i = d; // não compila
```

CASTING E PROMOÇÃO

Apesar de 5 ser um bom valor para um int , o compilador não tem como saber que valor estará dentro desse double no momento da execução. Esse valor pode ter sido digitado pelo usuário, e ninguém vai garantir que essa conversão ocorra sem perda de valores.
Já no caso a seguir, é o contrário:

```
int i = 5;
double d2 = i;
```

O código acima compila sem problemas, já que um double pode guardar um número com ou sem ponto flutuante. Todos os inteiros representados por uma variável do tipo int podem ser guardados em uma variável double, então não existem problemas no código acima.

Às vezes, precisamos que um número quebrado seja arredondado e armazenado num número inteiro.

CASTING E PROMOÇÃO

Para fazer isso sem que haja o erro de compilação, é preciso ordenar que o número quebrado seja moldado (casted) como um número inteiro. Esse processo recebe o nome de casting.

```
double d3 = 3.14;
int i = (int) d3;
```

O casting foi feito para moldar a variável d3 como um int . O valor de i agora é 3.
O mesmo ocorre entre valores int e long .

```
long x = 10000;
int i = x; // não compila, pois pode estar perdendo informação
```

E, se quisermos realmente fazer isso, fazemos o casting:

```
long x = 10000;
int i = (int) x;
```

CASTINGS POSSÍVEIS

Abaixo estão relacionados todos os casts possíveis na linguagem Java, mostrando a conversão de um valor para outro. A indicação Impl. quer dizer que aquele cast é implícito e automático, ou seja, você não precisa indicar o cast explicitamente (lembrando que o tipo boolean não pode ser convertido para nenhum outro tipo).

PARA: / DE:	byte	short	char	int	long	float	double
byte	----	*Impl.*	(char)	*Impl.*	*Impl.*	*Impl.*	*Impl.*
short	(byte)	----	(char)	*Impl.*	*Impl.*	*Impl.*	*Impl.*
char	(byte)	(short)	----	*Impl.*	*Impl.*	*Impl.*	*Impl.*
int	(byte)	(short)	(char)	----	*Impl.*	*Impl.*	*Impl.*
long	(byte)	(short)	(char)	(int)	----	*Impl.*	*Impl.*
float	(byte)	(short)	(char)	(int)	(long)	----	*Impl.*
double	(byte)	(short)	(char)	(int)	(long)	(float)	----

TAMANHO DOS TIPOS

Na tabela abaixo, estão os tamanhos de cada tipo primitivo do Java.

TIPO	TAMANHO
boolean	1 bit
byte	1 byte
short	2 bytes
char	2 bytes
int	4 bytes
float	4 bytes
long	8 bytes
double	8 bytes

EXERCÍCIOS

1) Na empresa onde trabalhamos, há tabelas com o quanto foi **gasto em cada mês**. Para fechar o balanço do primeiro trimestre, precisamos somar o gasto total. Sabendo que, em Janeiro, foram gastos 15000 reais, em Fevereiro, 23000 reais e em Março, 17000 reais, faça um programa que calcule e imprima o gasto total no trimestre. Siga esses passos:

- Crie uma classe chamada BalancoTrimestral com um bloco main, como nos exemplos anteriores;
- Dentro do main (o miolo do programa), declare uma variável inteira chamada gastosJaneiro e inicialize-a com 15000;
- Crie também as variáveis gastosFevereiro e gastosMarco , inicializando-as com 23000 e 17000, respectivamente, utilize uma linha para cada declaração;
- Crie uma variável chamada gastosTrimestre e inicialize-a com a soma das outras 3 variáveis: **int gastosTrimestre = gastosJaneiro + gastosFevereiro + gastosMarco**;
- Imprima a variável gastosTrimestre .

2) Adicione código (sem alterar as linhas que já existem) na classe anterior para imprimir **a média mensal de gasto**, criando uma variável mediaMensal junto com uma mensagem. Para isso, concatene a String com o valor, usando "Valor da média mensal = " + mediaMensal.

3) Crie um programa que exiba a **tabuada** de um número informado pelo usuário;

4) Crie um programa que **troque os valores** que estão armazenados nas variáveis nome1 e nome2.

- A variável nome1 armazenara o valor “mestre do universo”.
- A variável nome2 receberá um valor informado pelo usuário.
- Altere os valores das variáveis e exiba a soma da variável nome1 com o valor da variável nome2.

5) Crie um programa que **calcule a média de um aluno** para a matéria de matemática. O aluno deve informar seu nome e as quatro notas da matéria;

6) Escreva um programa que, com base em uma **temperatura** em graus celsius, a converta e exiba em Kelvin (K), Réaumur (Re), Rankine (Ra) e Fahrenheit (F), seguindo as fórmulas: F = C * 1.8 + 32; K = C + 273.15; Re = C * 0.8;
Ra = C * 1.8 + 32 + 459.67

7) Calcular e apresentar o valor do volume de uma **lata de óleo**, utilizando fórmula: V = 3.14159 * R * R * A, em que as variáveis: V, R e A representam respectivamente o volume, o raio e a altura. Vamos declarar as variáveis V, R e A como double e definir um valor qualquer para as variáveis de altura e de raio. Agora vamos escrever a fórmula, conforme dita no enunciado e, em seguida, exibir na tela uma mensagem contendo o resultado do volume encontrado.

Material Complementar

Problemas quando programava na década de 1990

- *Ponteiros;*
- gerenciamento de memória;
- *Organização;*
- Falta de bibliotecas;
- *Ter de reescrever parte do código ao mudar de sistema operacional;*
- Custo financeiro de usar a tecnologia.

Problemas quando programava na década de 1990

A linguagem Java resolve bem esses problemas, que até então apareciam com frequência nas outras linguagens. Alguns desses problemas foram particularmente atacados porque uma das grandes motivações para a criação da plataforma Java era de que essa linguagem fosse usada em pequenos dispositivos, como tvs, videocassetes, aspiradores, liquidificadores e outros. Apesar disso a linguagem teve seu lançamento focado no uso em clientes web (browsers) para rodar pequenas aplicações (applets). Hoje em dia esse não é o grande mercado do Java: apesar de ter sido idealizado com um propósito e lançado com outro, o Java ganhou destaque no lado do servidor.

Origem do Java

O Java foi criado pela antiga Sun Microsystems e mantida através de um comitê (http://www.jcp.org). Seu site principal era o java.sun.com, e java.com um site mais institucional, voltado ao consumidor de produtos e usuários leigos, não desenvolvedores. Com a compra da Sun pela Oracle em 2009, muitas URLs e nomes tem sido trocados para refletir a marca da Oracle. A página principal do Java é: http://www.oracle.com/technetwork/java/

UMA BREVE HISTÓRIA DO JAVA

A Sun criou um time (conhecido como Green Team) para desenvolver inovações tecnológicas em 1992. Esse time foi liderado por James Gosling, considerado o pai do Java. O time voltou com a ideia de criar um interpretador (já era uma máquina virtual, veremos o que é isso mais a frente) para pequenos dispositivos, facilitando a reescrita de software para aparelhos eletrônicos, como vídeo cassete, televisão e aparelhos de TV a cabo.

A ideia não deu certo. Tentaram fechar diversos contratos com grandes fabricantes de eletrônicos, como Panasonic, mas não houve êxito devido ao conflito de interesses e custos. Hoje, sabemos que o Java domina o mercado de aplicações para celulares com mais de 2.5 bilhões de dispositivos compatíveis, porém em 1994 ainda era muito cedo para isso.

UMA BREVE HISTÓRIA DO JAVA

Com o advento da web, a Sun percebeu que poderia utilizar a ideia criada em 1992 para rodar pequenas aplicações dentro do browser. A semelhança era que na internet havia uma grande quantidade de sistemas operacionais e browsers, e com isso seria grande vantagem poder programar numa única linguagem, independente da plataforma. Foi aí que o Java 1.0 foi lançado: focado em transformar o browser de apenas um cliente magro (thin client ou terminal burro) em uma aplicação que possa também realizar operações avançadas, e não apenas renderizar html.

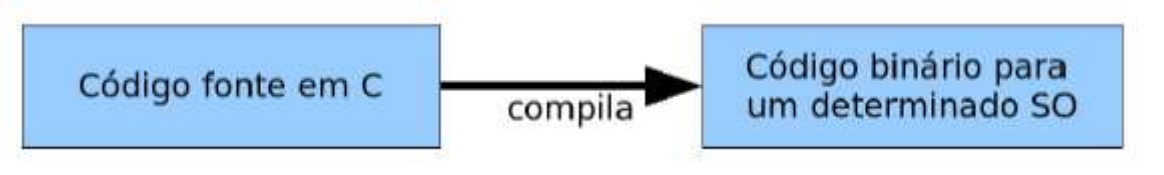
Os applets deixaram de ser o foco da Sun, e nem a Oracle nunca teve interesse. É curioso notar que a tecnologia Java nasceu com um objetivo em mente, foi lançado com outro, mas, no final, decolou mesmo no desenvolvimento de aplicações do lado do servidor.

UMA BREVE HISTÓRIA DO JAVA

Em 2009 a Oracle comprou a Sun, fortalecendo a marca. A Oracle sempre foi, junto com a IBM, uma das empresas que mais investiram e fizeram negócios através do uso da plataforma Java. Em 2014 surge a versão Java 8 com mudanças interessantes na linguagem.

MÁQUINA VIRTUAL

Em uma linguagem de programação como C e Pascal, temos a seguinte situação quando vamos compilar um programa:

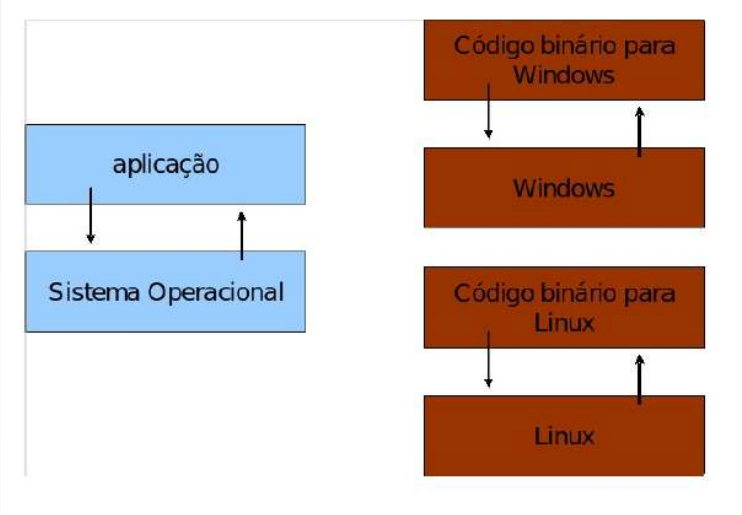


O código fonte é compilado para código de máquina específico de uma plataforma e sistema operacional. Muitas vezes o próprio código fonte é desenvolvido visando uma única plataforma!

MÁQUINA VIRTUAL

Esse código executável (binário) resultante será executado pelo sistema operacional e, por esse motivo, ele deve saber conversar com o sistema operacional em questão.

Isto é, temos um código executável para cada sistema operacional. É necessário compilar uma vez para Windows, outra para o Linux, e assim por diante, caso a gente queira que esse nosso software possa ser utilizado em várias plataformas. Esse é o caso de aplicativos como o OpenOffice, Firefox e outros.



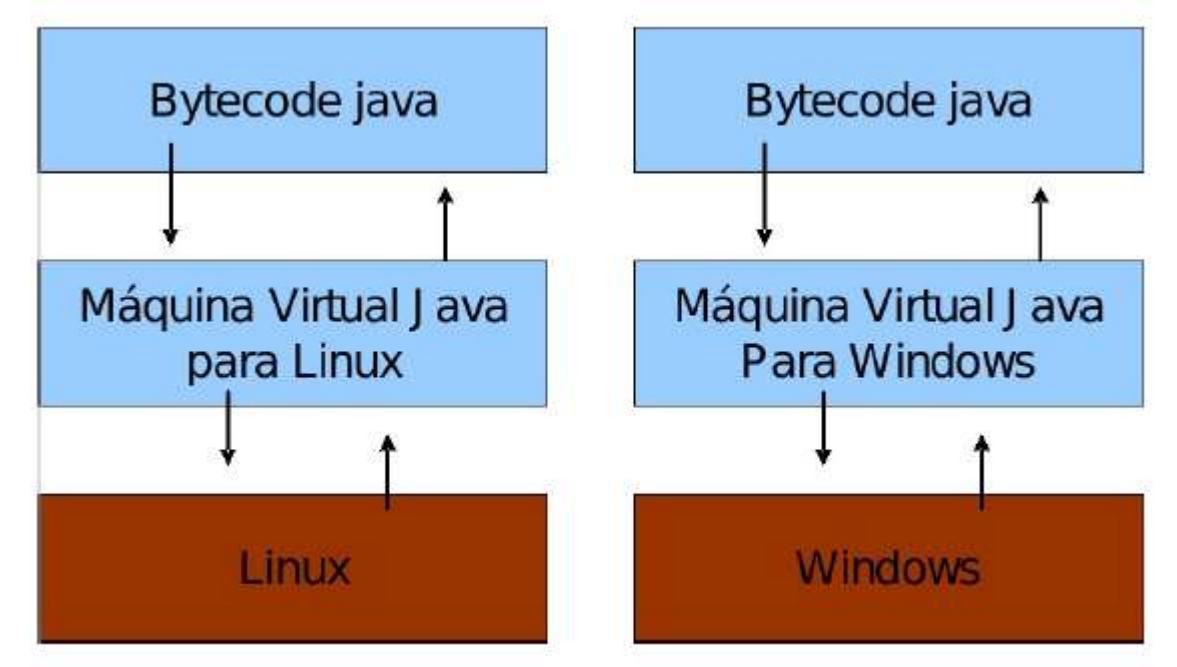
MÁQUINA VIRTUAL

Como foi dito anteriormente, na maioria das vezes, a sua aplicação se utiliza das bibliotecas do sistema operacional, como, por exemplo, a de interface gráfica para desenhar as "telas". A biblioteca de interface gráfica do Windows é bem diferente das do Linux: como criar então uma aplicação que rode de forma parecida nos dois sistemas operacionais?

Precisamos reescrever um mesmo pedaço da aplicação para diferentes sistemas operacionais, já que
eles não são compatíveis.

MÁQUINA VIRTUAL

Já o Java utiliza do conceito de máquina virtual, onde existe, entre o sistema operacional e a aplicação, uma camada extra responsável por "traduzir" - mas não apenas isso - o que sua aplicação deseja fazer para as respectivas chamadas do sistema operacional onde ela está rodando no momento:



MÁQUINA VIRTUAL

Dessa forma, a maneira com a qual você abre uma janela no Linux ou no Windows é a mesma: você ganha independência de sistema operacional. Ou, melhor ainda, independência de plataforma em geral:
não é preciso se preocupar em qual sistema operacional sua aplicação está rodando, nem em que tipo de máquina, configurações, etc.

Repare que uma máquina virtual é um conceito bem mais amplo que o de um interpretador. Como o próprio nome diz, uma máquina virtual é como um "computador de mentira": tem tudo que um computador tem. Em outras palavras, ela é responsável por gerenciar memória, threads, a pilha de execução, etc.

MÁQUINA VIRTUAL

Sua aplicação roda sem nenhum envolvimento com o sistema operacional! Sempre conversando apenas com a Java Virtual Machine (JVM).

Essa característica é interessante: como tudo passa pela JVM, ela pode tirar métricas, decidir onde é melhor alocar a memória, entre outros. Uma JVM isola totalmente a aplicação do sistema operacional.
Se uma JVM termina abruptamente, só as aplicações que estavam rodando nela irão terminar: isso não afetará outras JVMs que estejam rodando no mesmo computador, nem afetará o sistema operacional.

Essa camada de isolamento também é interessante quando pensamos em um servidor que não pode se sujeitar a rodar código que possa interferir na boa execução de outras aplicações.

MÁQUINA VIRTUAL

Essa camada, a máquina virtual, não entende código java, ela entende um código de máquina específico. Esse código de máquina é gerado por um compilador java, como o javac, e é conhecido por "bytecode", pois existem menos de 256 códigos de operação dessa linguagem, e cada "opcode" gasta um byte. O compilador Java gera esse bytecode que, diferente das linguagens sem máquina virtual, vai servir para diferentes sistemas operacionais, já que ele vai ser "traduzido" pela JVM.

WRITE ONCE, RUN ANYWHERE

Esse era um slogan que a Sun usava para o Java, já que você não precisa reescrever partes da sua aplicação toda vez que quiser mudar de sistema operacional.

JVM? JRE? JDK?

- **JVM** = apenas a virtual machine, esse download não existe, ela sempre vem acompanhada.
- **JRE** = Java Runtime Environment, ambiente de execução Java, formado pela JVM e bibliotecas,
- tudo que você precisa para executar uma aplicação Java. Mas nós precisamos de mais.
- **JDK** = Java Development Kit: Nós, desenvolvedores, faremos o download do JDK do Java SE (Standard Edition). Ele é formado pela JRE somado a ferramentas, como o compilador.

ONDE USAR E OS OBJETIVOS DO JAVA

É preciso ficar claro que a premissa do Java não é a de criar sistemas pequenos, onde temos um ou dois desenvolvedores, mais rapidamente que linguagens como php, perl, e outras.

O foco da plataforma é outro: aplicações de médio a grande porte, onde o time de desenvolvedores tem várias pessoas e sempre pode vir a mudar e crescer. Não tenha dúvidas que criar a primeira versão de uma aplicação usando Java, mesmo utilizando IDEs e ferramentas poderosas, será mais trabalhoso que muitas linguagens script ou de alta produtividade. Porém, com uma linguagem orientada a objetos e madura como o Java, será extremamente mais fácil e rápido fazer alterações no sistema, desde que você siga as boas práticas e recomendações sobre design orientado a objetos.

ONDE USAR E OS OBJETIVOS DO JAVA

Além disso, a quantidade enorme de bibliotecas gratuitas para realizar os mais diversos trabalhos (tais como relatórios, gráficos, sistemas de busca, geração de código de barra, manipulação de XML, tocadores de vídeo, manipuladores de texto, persistência transparente, impressão, etc) é um ponto fortíssimo para adoção do Java: você pode criar uma aplicação sofisticada, usando diversos recursos, sem precisar comprar um componente específico, que costuma ser caro. O ecossistema do Java é enorme.

ESPECIFICAÇÃO VERSUS IMPLEMENTAÇÃO

Outro ponto importante: quando falamos de Java Virtual Machine, estamos falando de uma especificação. Ela diz como o bytecode deve ser interpretado pela JVM. Quando fazemos o download no site da Oracle, o que vem junto é a Oracle JVM. Em outras palavras, existem outras JVMs disponíveis, como a JRockit da BEA (também adquirida pela Oracle), a J9 da IBM, entre outras.

Esse é outro ponto interessante para as empresas. Caso não estejam gostando de algum detalhe da JVM da Oracle ou prefiram trabalhar com outra empresa, pagando por suporte, elas podem trocar de JVM com a garantia absoluta de que todo o sistema continuará funcionando. Isso porque toda JVM deve ser certificada pela Oracle, provando a sua compatibilidade. Não há nem necessidade de recompilar nenhuma de suas classes.